Why Keycloak

I decided to use Keycloak since it's a mature, open source solution for running an Authorization Server yourself. After increasing demands from the community, Spring started a new Spring Authorization Server project (https://github.com/spring-projects/spring-authorization-server). Since version 0.2.0, it's been a production-ready solution for setting up an OAuth2 Authorization Server. At the time of writing, the project provides an implementation for the most common OAuth2 features, and it's currently working on extending support for OIDC-specific features. You can follow the progress and contribute to the project on GitHub.

Another option is to use a SaaS solution like Okta (www.okta.com) or Auth0 (https://auth0.com). They are both excellent solutions for getting OIDC/OAuth2 as a managed service, and I encourage you to try them out. For this book, I wanted to use a solution that you could run and reliably reproduce in your local environment without depending on other services that might change over time, making my instructions here invalid.

Before moving on, let's stop any running containers. Open a Terminal window, navigate to the folder where you keep your Docker Compose file (polar-deployment/ docker/docker-compose.yml), and run the following command:

```
$ docker-compose down
```

We now have all the pieces for refactoring Edge Service so it can use an authentication strategy relying on OIDC/OAuth2, JWT, and Keycloak. The best part is that it's based on standards and supported by all major languages and frameworks (frontend, backend, mobile, IoT), including Spring Security.

11.4 Authenticating users with Spring Security and OpenID Connect